

AWS Hospital Care Notification System

Step-by-Step Project Documentation

Project focus: AWS VPC + private EC2 + IAM + VPC Endpoints + Amazon SNS

Project Objective

This project demonstrates how to run an EC2 application/server inside a custom AWS VPC without a public IPv4 address and securely allow it to communicate with AWS services using VPC endpoints, IAM, and Amazon SNS. A test notification is published from the EC2 instance using the AWS CLI and the SNS topic has a confirmed email subscription.

Outcome: A private EC2 instance can publish an SNS notification without a public IPv4 address, using AWS service connectivity through VPC endpoints and permissions supplied by an IAM role.

Prepared from the provided AWS console and terminal screenshots

1. Project Overview and Architecture

The solution is designed around a private EC2 instance inside a custom VPC. The instance does not have a public IPv4 address. AWS service access is provided through VPC endpoints, while IAM controls what the instance is allowed to do. Amazon SNS is used for notification delivery.

Component	Configuration / Purpose
VPC	hospital-care-vpc — IPv4 CIDR 10.0.0.0/16
EC2	hospital-care — t3.micro in a private subnet; private IP 10.0.131.174
IAM Role	Hospital-Report — used by the EC2 instance
IAM Policies	AmazonSNSFullAccess and AmazonSSMManagedInstanceCore
VPC Endpoints	S3 Gateway + SNS, SSM, SSM Messages, EC2 Messages interface endpoints
SNS	hospital-care — Standard topic with confirmed email subscription
Test	AWS CLI sns publish from the EC2 instance; MessageId returned successfully

Logical flow

EC2 (private subnet) → IAM role permissions → VPC endpoint for SNS → Amazon SNS topic → confirmed email subscription.

Management path: EC2 → SSM-related VPC endpoints → AWS Systems Manager. The project also includes an S3 Gateway endpoint.

Step 1 — Create and verify the VPC

Create a dedicated VPC for the Hospital Care environment. The project VPC is named hospital-care-vpc and uses the private IPv4 range 10.0.0.0/16. DNS resolution and DNS hostnames are enabled, which supports AWS service communication and private name resolution.

vpc-009e6b49231c43814 / hospital-care-vpc Actions ▾

Details Info

VPC ID vpc-009e6b49231c43814	State Available	Block Public Access Off	DNS hostnames Enabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-08422a82a3844a218	Main route table rtb-0669675feab374b5a
Main network ACL acl-04f3a0a5707fdb65d	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group) -	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 905367996208
Encryption control ID -	Encryption control mode -		

Figure 1. Custom VPC details.

Step 2 — Create the IAM role for EC2

Create/attach an IAM role named Hospital-Report and associate it with the EC2 instance. The screenshot shows two AWS managed policies: AmazonSNSFullAccess, which permits SNS operations, and AmazonSSMManagedInstanceCore, which allows Systems Manager management of the instance. Using an instance role avoids putting long-term AWS access keys on the server.

Summary

Creation date
August 02, 2026, 16:17 (UTC+05:30)

ARN
arn:aws:iam::905367996208:role/Hospital-Report

Instance profile ARN
arn:aws:iam::905367996208:instance-profile/Hospital-Report

Last activity
6 minutes ago

Maximum session duration
1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (2) Info

Refresh Simulate Remove Add permissions ▾

Filter by Type
All types

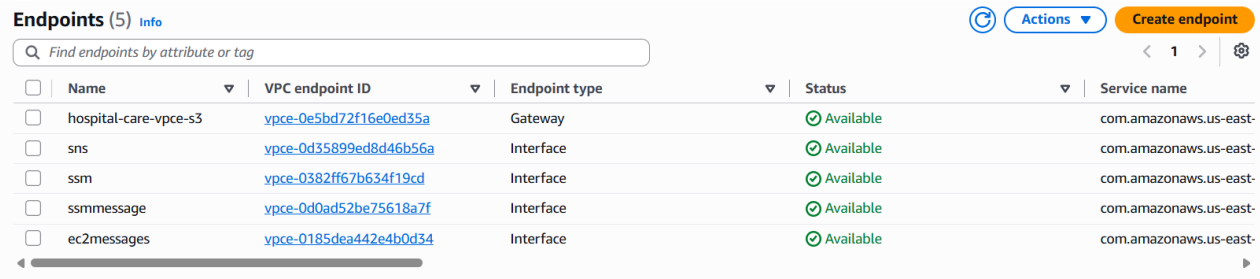
< 1 > Settings

☐	Policy name	Type	Attached entities
☐	AmazonSNSFullAccess	AWS managed	1
☐	AmazonSSMManagedInstanceCore	AWS managed	1

Figure 2. IAM role permissions.

Step 3 — Configure VPC endpoints

Because the EC2 instance is in a private subnet and has no public IPv4 address, VPC endpoints provide private connectivity to AWS services. The screenshot shows five endpoints: an S3 Gateway endpoint and Interface endpoints for SNS, SSM, SSM Messages, and EC2 Messages. The endpoints are shown as Available.

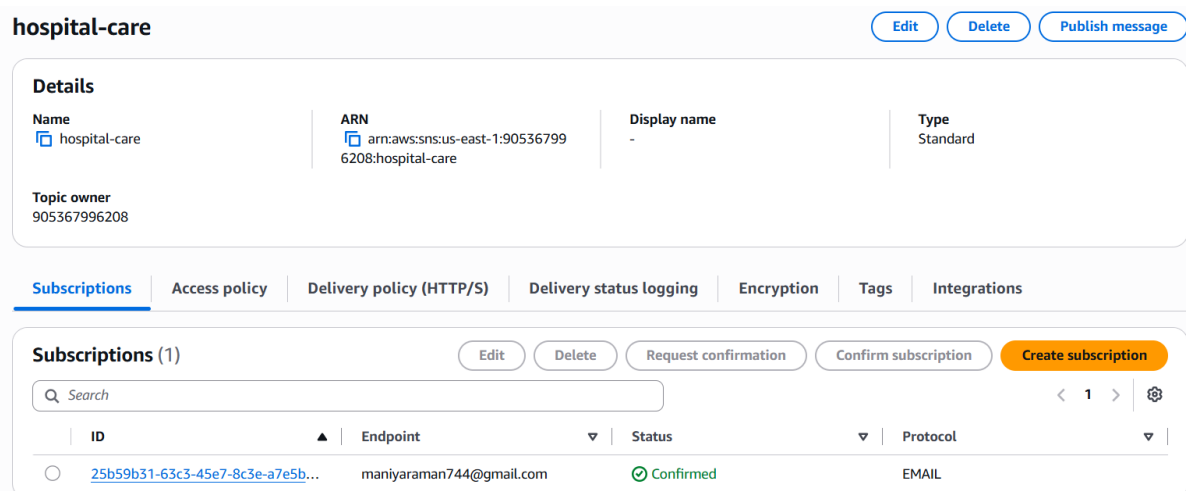


Endpoints (5) Info						Actions	Create endpoint
Find endpoints by attribute or tag						< 1 >	
☐	Name	VPC endpoint ID	Endpoint type	Status	Service name		
☐	hospital-care-vpce-s3	vpce-0e5bd72f16e0ed35a	Gateway	Available	com.amazonaws.us-east-		
☐	sns	vpce-0d35899ed8d46b56a	Interface	Available	com.amazonaws.us-east-		
☐	ssm	vpce-0382ff67b634f19cd	Interface	Available	com.amazonaws.us-east-		
☐	ssmmessages	vpce-0d0ad52be75618a7f	Interface	Available	com.amazonaws.us-east-		
☐	ec2messages	vpce-0185dea442e4b0d34	Interface	Available	com.amazonaws.us-east-		

Figure 3. VPC endpoints used by the project.

Step 4 — Create the SNS topic and subscription

Create a Standard Amazon SNS topic named hospital-care. Add an email subscription and confirm the subscription from the email inbox. The project screenshot shows the subscription endpoint as maniyaram...@gmail.com and its status as Confirmed. Once confirmed, the topic can deliver published messages to that email endpoint.



hospital-care Edit Delete Publish message

Details

Name hospital-care	ARN arn:aws:sns:us-east-1:90536799:6208:hospital-care	Display name -	Type Standard
------------------------------	---	--------------------------	-------------------------

Topic owner
905367996208

Subscriptions Access policy Delivery policy (HTTP/S) Delivery status logging Encryption Tags Integrations

Subscriptions (1) Edit Delete Request confirmation Confirm subscription Create subscription

Search

ID	Endpoint	Status	Protocol
25b59b31-63c3-45e7-8c3e-a7e5b...	maniyaraman744@gmail.com	Confirmed	EMAIL

Figure 4. SNS topic and confirmed subscription.

Step 5 — Launch and verify the EC2 instance

Launch the EC2 instance in the private subnet and attach the Hospital-Report IAM role. The instance shown is t3.micro with private IPv4 address 10.0.131.174. It has no public IPv4 address or public DNS name, confirming that it is operating without direct Internet exposure. The instance is in the hospital-care-vpc VPC and uses the private subnet shown in the screenshot.

Instance summary for i-06a1a7beb3ef819b6 (hospital-care) [Info](#)

Updated less than a minute ago

Instance ID
[i-06a1a7beb3ef819b6](#)

IPv6 address
—

Hostname type
IP name: ip-10-0-131-174.ec2.internal

Answer private resource DNS name
—

Auto-assigned IP address
—

IAM role
[Hospital-Report](#)

IMDSv2
Required

Public IPv4 address
—

Instance state
Running

Private IP DNS name (IPv4 only)
[ip-10-0-131-174.ec2.internal](#)

Instance type
t3.micro

VPC ID
[vpc-009e6b49231c43814 \(hospital-care-vpc\)](#)

Subnet ID
[subnet-0dbd8772562db11d7 \(hospital-care-subnet-private1-us-east-1a\)](#)

Instance ARN
[arn:aws:ec2:us-east-1:905367996208:instance/i-06a1a7beb3ef819b6](#)

Private IPv4 addresses
[10.0.131.174](#)

Public DNS
—

Elastic IP addresses
—

AWS Compute Optimizer finding
[Opt-in to AWS Compute Optimizer for recommendations.](#)
[Learn more](#)

Auto Scaling Group name
—

Managed
false

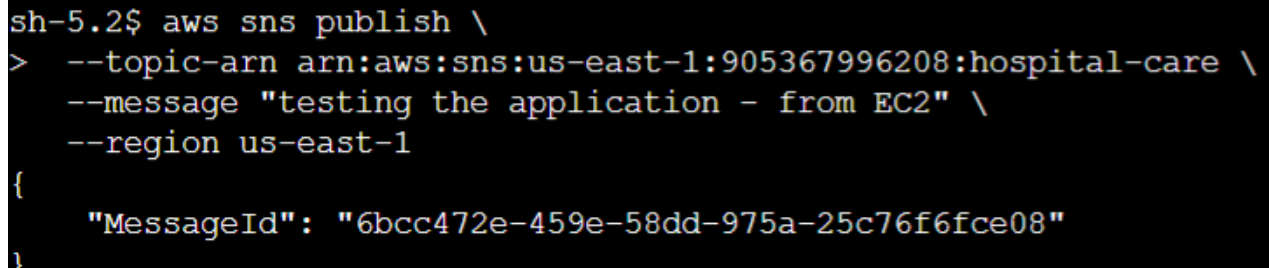
Figure 5. EC2 instance running in the private subnet.

Step 6 — Publish a test SNS message from EC2

From the EC2 shell, use the AWS CLI to publish a test message to the hospital-care SNS topic. The command shown uses the topic ARN and the us-east-1 region. AWS returns a MessageId, which confirms that the publish API request succeeded.

AWS CLI command used

```
aws sns publish \ --topic-arn arn:aws:sns:us-east-1:905367996208:hospital-care \ --message  
"testing the application - from EC2" \ --region us-east-1
```



```
sh-5.2$ aws sns publish \  
> --topic-arn arn:aws:sns:us-east-1:905367996208:hospital-care \  
--message "testing the application - from EC2" \  
--region us-east-1  
{  
  "MessageId": "6bcc472e-459e-58dd-975a-25c76f6fce08"  
}
```

Figure 6. Successful AWS CLI SNS publish.

The returned JSON contains a MessageId. This is the key evidence that the SNS publish request was accepted by Amazon SNS.

7. Verification and Expected Result

Use the following checklist after completing the configuration. These points match the state visible in the supplied screenshots.

- VPC status: Available; CIDR 10.0.0.0/16.
- EC2 instance: Running; t3.micro; private IP 10.0.131.174; no public IPv4 address.
- IAM role: Hospital-Report attached to the EC2 instance.
- IAM permissions: AmazonSNSFullAccess and AmazonSSMManagedInstanceCore.
- VPC endpoints: all shown as Available.
- SNS topic: hospital-care; Standard topic.
- SNS subscription: Email subscription shown as Confirmed.
- SNS test: AWS CLI publish returned a MessageId successfully.

Expected notification flow

1. The EC2 instance runs in the private subnet.
2. The AWS CLI uses the EC2 instance role credentials.
3. The request reaches SNS through the SNS VPC interface endpoint.
4. SNS accepts the message and returns a MessageId.
5. SNS delivers the message to the confirmed email subscription.

8. Troubleshooting and Project Summary

Common troubleshooting

SNS publish fails with AccessDenied

Check that the EC2 instance is using the Hospital-Report role and that the role has the required SNS permission.

SNS publish times out from a private subnet

Check the SNS interface VPC endpoint, its security group, route/DNS settings, and that the endpoint is Available.

SSM cannot connect to the instance

Verify AmazonSSMManagedInstanceCore is attached and the SSM, SSM Messages, and EC2 Messages endpoints are available.

Email notification is not received

Check the SNS subscription status. If it is PendingConfirmation, confirm the subscription from the email.

AWS CLI cannot find the topic

Verify the topic ARN and region. The screenshots show the project using us-east-1.

Security notes

Keep the EC2 instance private when public access is not required. Prefer IAM roles over storing AWS access keys on the instance. Limit IAM permissions to only the actions required by the application when moving from a training project to production. Review VPC endpoint security groups and SNS topic access policies before production use.

Conclusion

The completed project demonstrates a practical AWS cloud architecture in which a private EC2 instance communicates with Amazon SNS through VPC endpoints. IAM provides identity and permissions, while SNS handles notification delivery. The successful CLI publish and returned MessageId provide the final verification of the end-to-end configuration.